CO2039: Advanced Programming

Lambda Calculus
An introduction (cont.')

Disclaimer: The images in these slides are AI-generated.

Learning outcomes

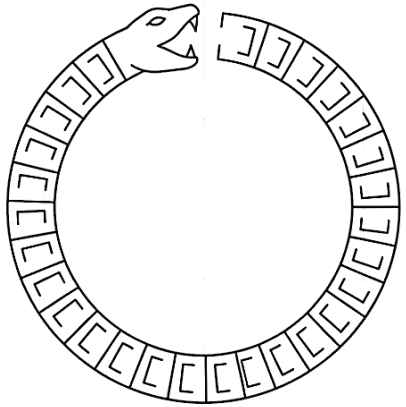
By the end of this lecture, you will be able to:

- Explain how recursion emerges from fixed points
- Describe how the Y-combinator enables self-reference
- Distinguish call-by-value and call-by-name evaluation
- Compare untyped and simply typed lambda calculus
- Explain the Church-Turing thesis
- Relate lambda calculus to modern programming paradigms

The primitive world of λ-calculus

Variables	Abstraction	Application
`x, y, z`	`λx.M`	`M N`
Placeholders for values.	Function creation.	Function application.

The paradox: In a world of anonymous functions, how can a function refer to itself to create a loop?



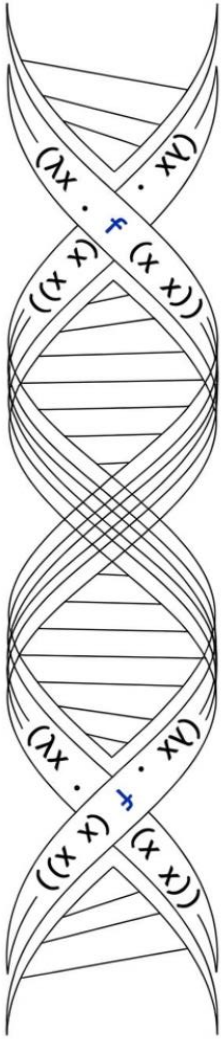
The fixed-point idea

A fixed-point of a function F is a value X such that

$$F(X) = X$$

If we can mathematically construct this X, we unlock recursion.

The Y-combinator

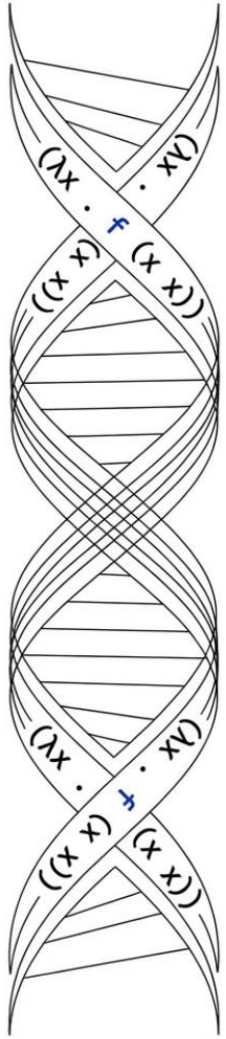


$$Y = \lambda f.(\lambda x.f(xx))\ (\lambda x.f(xx))$$

The pattern replicator:

$$Y\ F \rightarrow_{\beta} F\ (Y\ F)$$

Y generates recursion without naming.
It allows F to operate on itself infinitely



Y in action: The factorial function

1. Define Logic (F)

$$F = \lambda f.\ \lambda n.\ (\text{if } n=0 \text{ then } 1 \text{ else } n \cdot f(n-1))$$

Logic exists, but 'f' is just a variable.

↓

2. Apply Y Combinator

$$Fact = Y\ F$$

Y supplies the recursion mechanism.

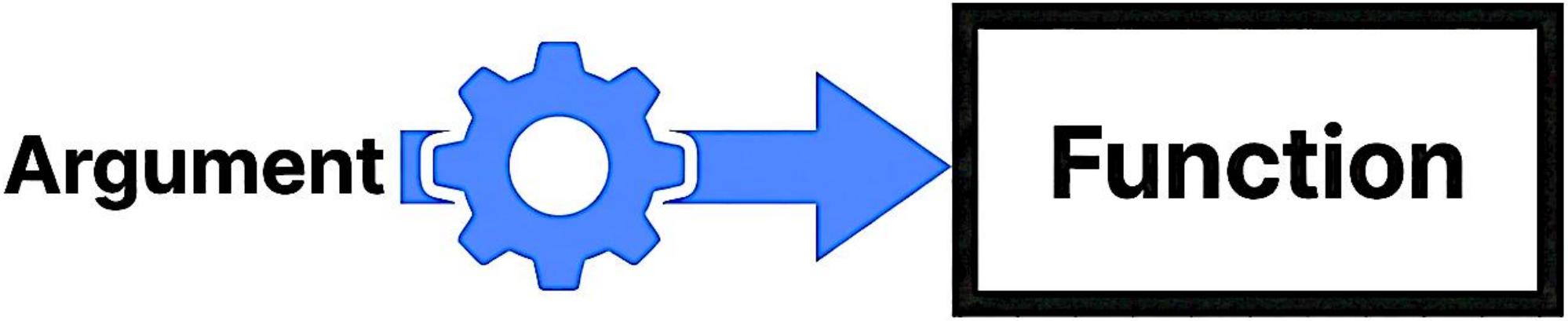
↓

3. Result

$$Fact(n) = n \cdot Fact(n-1)$$

A working recursive function.

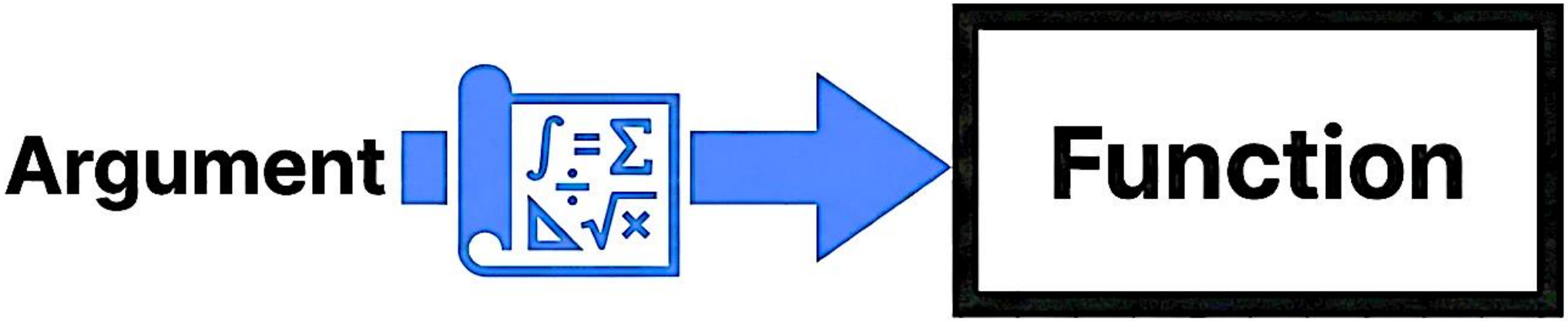
Strategy: Call-by-value (Eager evaluation)



Evaluate the argument completely **before** passing it into the function

Used by: C, Java, Python

Strategy: Call-by-name (Lazy evaluation)



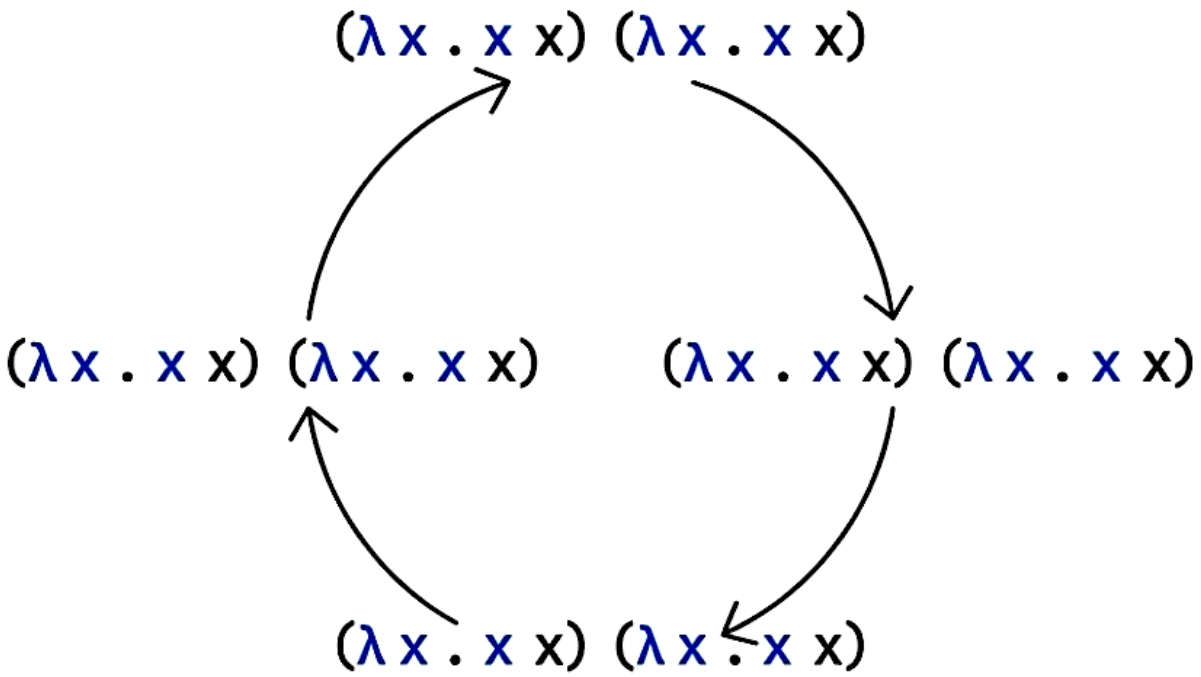
Pass the expression **unevaluated**. Compute **only** if used.

Used by: Haskell.

Can handle infinite data structures.

The danger of absolute freedom

The divergent term reduces to itself forever:

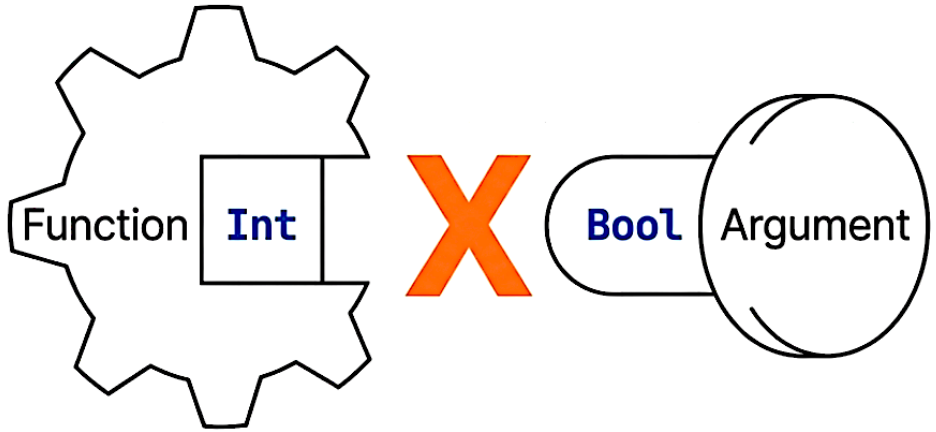


In untyped λ-calculus, anything can be applied to anything.

Result: No safety. No termination guarantee.

Simply Typed Lambda Calculus (STLC)

$$\text{Types } (\tau) ::= \textbf{Bool} \mid \textbf{Nat} \mid \tau \to \tau$$



Everything must have a type.
If the types don’t match, the program is rejected.

How we enforce order

Abstraction Rule

$$\frac{\Gamma, x{:}\tau_1 \vdash M{:}\tau_2}{\Gamma \vdash \lambda x.M : \tau_1 \to \tau_2}$$

If input matches τ_1 and body matches τ_2, the function is $\tau_1 \to \tau_2$.

Application Rule

$$\frac{\Gamma \vdash M : \tau_1 \to \tau_2 \qquad \Gamma \vdash N : \tau_1}{\Gamma \vdash M\ N : \tau_2}$$

You can only apply a function if the input type matches exactly.

The benefits of types

1. Type Safety

Well-typed programs do not “go wrong”.

(Reference: Progress & Preservation Theorem)

2. Strong Normalization

Every valid STLC program is guaranteed to terminate.

No infinite loops possible.

The great trade-off: Expressiveness vs. Safety

Untyped Lambda calculus	Simply Typed (STLC)
Recursion: YES (Y Combinator)	Recursion: NO (General Y is ill-typed)
Termination: NO (Divergence possible)	Termination: YES (Strong Normalization)

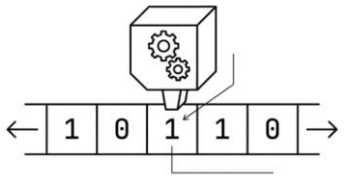
Insight: You cannot have unrestricted recursion AND guaranteed termination simultaneously.

The Curry-Howard Correspondence

LOGIC		PROGRAMMING
Propositions	↔	Types
Proofs	↔	Programs
Simplification	↔	Execution

Propositions are Types. Proofs are Programs.
If you can compile it, you have proven a theorem.

The Church-Turing Thesis



Church	Turing
Mathematical logic	Mechanical procedure
Functional view	Machine model
Abstract rewriting	Step-by-step execution

The thesis: Any effectively computable function can be computed by a Turing machine (equivalently, in λ–calculus).

Important:

- It is not a theorem (i.e. the meaning of computation)
- However, it can be proven that λ–calculus can simulate Turing machines and vice versa (hence, they are computationally equivalent)
- It is a foundational hypothesis about computation

From theory to practice

Imperative Paradigm

Root: Turing Machines

- Focus: State, Assignment, Loops

$f(x)$

Functional Paradigm

Root: λ-Calculus

- Focus: Expressions, Recursion, Immutable

Computability

Coding Factorial: C vs. Haskell

Imperative (C)

```
int fact(int n) {
  int r = 1;
  for (int i=1; i<=n; i++)
    r *= i;
  return r;
}
```

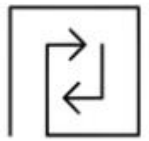
State Mutation

Functional (Haskell)

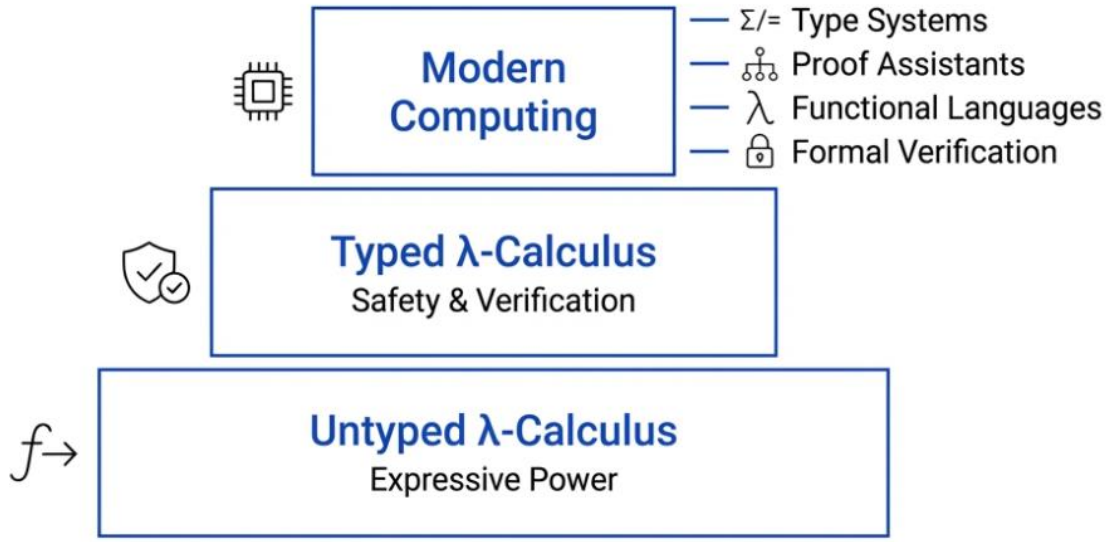
```
fact 0 = 1
fact n = n * fact (n-1)
```

Recursive Definition

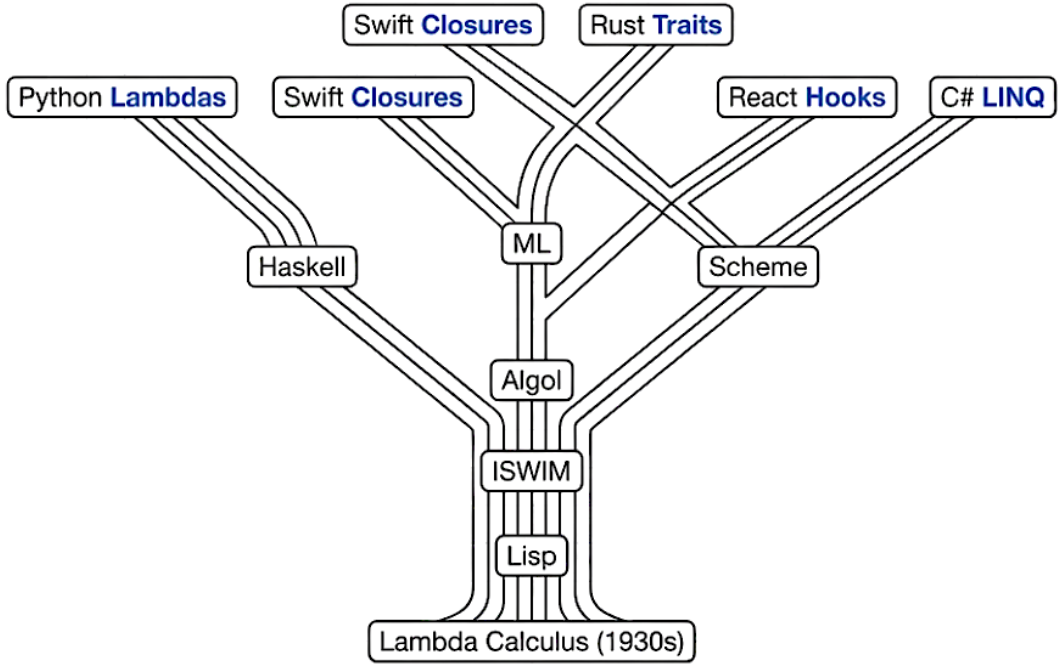
Thinking differently

Program as State Machine	Program as Mathematical Function
Ordered Commands	Evaluated Expressions
Mutable Variables (x = x + 1) 	Immutable Values (x is always x) =
Loops 	Recursion 

The legacy of Lambda



Lambda calculus is not just theory. It is the mathematical DNA of modern programming.



Every time you pass a function as an argument, you are using Lambda Calculus.

Takeaway messages

- Recursion in lambda calculus is constructed via fixed points.
- Absolute expressive freedom allows divergence.
- Types enforce discipline: safety and guaranteed termination.
- Church, Turing, Kleene describe the same notion of computability.
- Modern programming languages inherit ideas from lambda calculus.